

Assignment 3: Reddit Deep-Dive

due Monday 07/13 at 9:00am (EST)

Adapted from Cornell's CS 1110 (Spring 2025) materials

After finishing this assignment, submit the `assignment-3.py`, `assignment-3.ipynb`, and `assignment-3-logs.txt` files on Canvas.

Introduction

Let's recap!

In general, our assignments will consist of questions where AI usage is not allowed and those where AI use is expected. Questions where AI use is expected will be explicitly marked as such; if a question has no such label, assume that AI use is not permitted.

You will usually be expected to submit three files on Canvas for every assignment. These are: a `.py` file, where you will write any required code, a `.txt` file with any AI chatbot conversations related to the assignment and any other outside sources you use, and a `.ipynb` file that answers any written questions associated with the assignment.

For this assignment, as with all other work for this course, you are expected to follow the academic integrity guidelines in the course syllabus and Cornell's [Code of Academic Integrity](#).

General Tips + Guidelines

There are several tips and guidelines that you should follow for this assignment:

- You will want to run the code in `assignment-3.py` from the terminal. You can use VSCode's terminal or your computer's terminal app.
- You should run your code in the virtual environment we created in Lab 1. If you have trouble figuring out how to activate it, try Googling! If you're still stuck, reach out to me – the earlier the better.
- Run your code regularly as you write it. It's much easier to catch errors if you debug early and often! I highly recommend that you run the included tests for each task directly after writing the function they test.
- The assignment is broken down into many tasks and it's easy to skip one by accident. Before submitting, go through each task to make sure you've completed it.
- For this assignment, you shouldn't use, and don't need, any Python constructs not yet covered in class. You should demonstrate your skills with the Python we've learned so far!

Part 0

This assignment will ask you to analyze the structure of Reddit data; specifically, three posts and their associated comments.

Before beginning, make sure that you have downloaded the `assignment-3.pdf` instructions, `assignment-3.py`, and `assignment-3.ipynb` files and the zipped `reddit-data` folder. You should place everything in the `info1100/assignments` folder and unzip the `reddit-data` folder.

All code for this assignment will be written in `assignment-3.py`.

Task 1: Fill in the `# Author` and `# NetID` comments at the top of the `.py` file.

Part 1

As usual, the `assignment-3.py` file already has some code in it. This code asks the user to input the topic of one of the reddit posts – `bench-clearing`, `fisking`, or `tontine` — and then loads the data as a dictionary using the `load_json` function. The data is then saved in a variable named `data`. The code also includes a `convert_timestamp` function which you will use in the Part 1 tasks.

You will be asked to write code to fill in the provided function stubs and to make use of those functions in the `if __name__ == '__main__':` section. You'll be writing the code to test each function as you go!

Before you start writing code, I recommend you poke around at the JSON files containing the Reddit data to get a sense of their structure. You can do this by opening the files and looking at the data directly, or use the file loading code already in `assignment-3.py` and explore with Python.

Task 2: Write the `pretty_post` function. This function should take the dictionary representing the Reddit post as an argument and return a readable string version of the post. Specifically, the string should have the following format:

```
Title: [title]
Author: [author]
Date: [formatted date]

Text:
[post text]

Score: [score]
Upvote Ratio: [upvote ratio]
# Comments: [number of comments]
```

The date should be of the format `[month] [day], [year] [hour]:[minutes]`. If you pass

the timestamp included in the post data to the `convert_timestamp` function, it will return the date in this format, but I'll leave it to you to figure out which field stores the timestamp! Then, replace the `Write the Task 2 test here` comment with a call to the function and double check that your function acts as expected on all three of the Reddit file posts.

Task 3: We also want to be able to easily read the comment contents. To do this, fill in the `pretty_comment` function. This function should take the dictionary representing a comment and return a readable string version of the comment with the following format:

```
ID: [id]
Author: [author]
Date: [formatted date]

Body:
[comment body]

Parent ID: [parent ID]
Score: [score]
```

You should only include the part of the parent ID after the underscore. Again, you should use `convert_timestamp` to format the comment date.

Task 4: Now, let's print all top-level comments; that is, the comments that reply directly to the main post. Write the `pretty_top_comments` function, which takes the list of all comments as an argument, and prints the readable version of each. Between each comment, print a line of dashes. After you've written this function, replace the `Write the Task 4 test here` comment with a call to `pretty_top_comments` to check your work.

One of the most interesting things about the Reddit data is that the structure is inherently nested. That is, each comment may itself have any number of replies, each of which is another comment.

Task 5: One thing we might be interested in is conversation threads — chains of replies starting at a particular comment on the original post, and continuing until we hit a comment that doesn't have any replies to it. Let's look for the longest thread, or chain of replies, that originates at a comment on the original post. Write the `find_longest_thread` function, which takes in a list of comments and returns the length of the longest thread originating at one of those comments. You should write this function using for or while loops. This function is tricky to write, so here's some tips:

- Consider all comments that are n deep in a thread at the same time.
- Use a list to store all comments at the current level, or depth. Loop through them to gather comments that are one level deeper, or are replies to comments at the current depth.
- Continue going one level deeper in the threads until there are no comments at the next level.

- As you go through the levels, keep track of how deep you've gone!

Once you've finished writing this function, replace the `Write the Task 5 test here` comment with a call to `find_longest_thread` to check your work. The output for `bench-clearing` should be `4`, `fisking` should be `5`, and `tontine` should be `10`.

Task 6: Recursive functions are often well-suited to analyzing recursive data! Write the `find_longest_thread_recursive` function, which should do exactly what `find_longest_thread` does, but with recursion. Some tips for the recursive version of this function are:

- Your base case should be when there are no comments at that level of the thread.
- Compare the longest thread originating at all comments in the list passed as an argument to determine what the longest thread is.

Again, once you've finished writing this function, replace the `Write the Task 6 test here` comment with a call to `find_longest_thread_recursive` to check your work. The outputs for this function should be exactly the same as those for `find_longest_thread`.

Task 7: It'd also be useful to double-check how many comments in total are in the replies to each post. The post data claims to include it, but when we're using other people's data, it's always good to double-check! Write the `count_comments` function, which takes in a list of comments, and returns the total number of comments represented, including replies. Write this function using only `for` or `while` loops — no recursion! It will likely look a lot like your `find_longest_thread` function. Once you've written the function, replace the `Write the Task 7 test here` comment with a call to `count_comments` to check your work. The output for `bench-clearing` should be `7`, `fisking` should be `63`, and `tontine` should be `54`.

Task 8: Once again, the nested structure of this data makes recursive analysis a good fit! Write the `count_comments_recursive` function, which should do exactly what `count_comments` does, but use recursion. Again, I recommend taking inspiration from your `find_longest_thread_recursive` function! After writing `count_comments_recursive`, replace the `Write the Task 8 test here` comment with a call to it to check your work. The outputs should be the same as those for Task 7.

Part 2 (AI Free Choice)

These tasks will all ask you to use the AI chatbot of your choice. You should copy and paste every conversation you have with a chatbot related to this assignment into a text file called `assignment-3-logs.txt`. You're expected to understand on a high level any code you include in the assignment; you will be asked to describe all AI-generated code as part of the assignment. You may also be asked to explain or edit this code in oral assessments.

All written questions should be answered in the `assignment-3.ipynb` file.

Task 9: In the `assignment-3.ipynb` file, fill in the blanks that ask for your name, NetID,

and which AI chatbots you use. If you want to use multiple chatbots, specify which one is used for which question!

We've mostly looked at surface level features of the Reddit posts, but there's a lot that might be interesting in the texts of the posts and comments themselves! We might want to count how frequently certain pronouns are used (are people talking about themselves? others? inanimate objects), or the most common nouns (what *things* are people discussing)!

Task 10: Brainstorm some property of the post and comment texts you're interested in! I recommend it be about as complicated as the ideas I've proposed above; feel free to base your idea on what I've proposed, although you shouldn't copy exactly what I've suggested! Under the Task 10 heading in `assignment-3.ipynb`, describe your idea in a few sentences.

Task 11: Now, prompt an AI chatbot to execute the idea you brainstormed in Task 10. I recommend you add your new code as a contained function in the `assignment-3.py` file, then replace the `Write the Task 11 test here` comment with a call to your new function.

Task 12: You should prompt the chatbot until you understand broadly what the code you generated for Task 11 does. Under the Task 12 header in the `assignment-3.ipynb` file, describe your Task 11 code in a few sentences.